

JAVA LAB RECORD

Experiment No.	06
Date	23/09/26
Title	Implementation of HashSet Methods in Java

Objective / Aim

To study and implement commonly used methods of the Java HashSet class, including adding unique elements, checking for elements, finding the size, removing elements, checking whether the set is empty, and clearing all elements.

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Keyboard and mouse
- Monitor/display

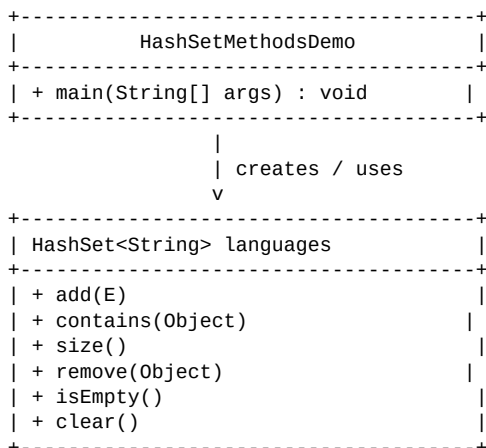
Software Requirements

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK): 17 or later
- IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Start the program.
2. Import the HashSet class from java.util.
3. Create a HashSet named languages to store String values.
4. Add Java, Python, and C++ using add(E e).
5. Attempt to add Java again and observe that HashSet does not store duplicate elements.
6. Check whether Java and HTML are present using contains(Object).
7. Find the number of elements using size().
8. Remove Python using remove(Object).
9. Check whether the HashSet is empty using isEmpty().
10. Remove all elements using clear().
11. Check the empty state again using isEmpty().
12. Display all results.
13. Stop the program.

UML Class Diagram



Source Code

```
import java.util.HashSet;

class HashSetMethodsDemo {

    public static void main(String[] args) {

        // Creating a HashSet
        HashSet<String> languages = new HashSet<>();

        // add(E e)
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");
        languages.add("Java");    // Duplicate, will not be added

        System.out.println("After add(): " + languages);

        // contains(Object o)
        System.out.println("Contains Java? " + languages.contains("Java"));
        System.out.println("Contains HTML? " + languages.contains("HTML"));

        // size()
        System.out.println("Size of HashSet: " + languages.size());

        // remove(Object o)
        languages.remove("Python");
        System.out.println("After remove(): " + languages);

        // isEmpty()
        System.out.println("Is HashSet empty? " + languages.isEmpty());

        // clear()
        languages.clear();
        System.out.println("After clear(): " + languages);

        // isEmpty() after clear
        System.out.println("Is HashSet empty now? " + languages.isEmpty());
    }
}
```

Completion and Execution Commands

1. Save the program as:
HashSetMethodsDemo.java
2. Open Command Prompt / Terminal and move to the folder containing the Java file.
3. Compile the program:
javac HashSetMethodsDemo.java
4. Execute the program:
java HashSetMethodsDemo
5. The results of all HashSet operations are displayed on the console.

Output

Note: HashSet does not guarantee a particular iteration order, so the order of elements shown in the first and removal outputs may vary.

```
After add(): [Java, C++, Python]
Contains Java? true
Contains HTML? false
Size of HashSet: 3
After remove(): [Java, C++]
Is HashSet empty? false
After clear(): []
Is HashSet empty now? true
```

Result

Thus, the Java program was successfully implemented and executed to demonstrate the commonly used methods of the HashSet class and its property of storing unique elements.